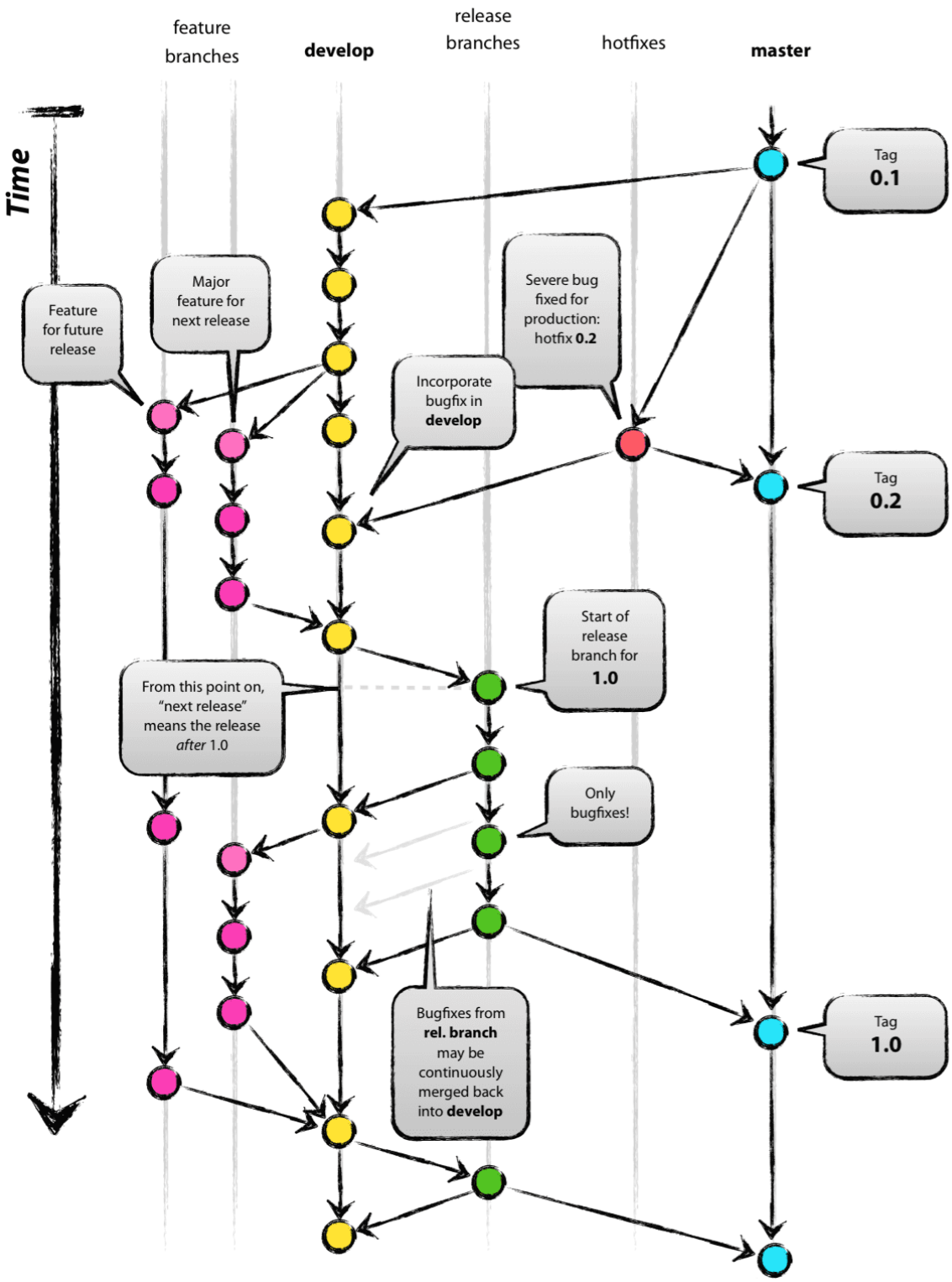
BEAM – BRANCHING STRATEGIES

Semantic Versioning & Git-flow



Overview

This lesson assumes you have a working knowledge of basic Git commands, so turn back now and review the previous lesson if you need to refresh your memory.

Version control is incredibly powerful, and now that you know some basic Git commands it is time to discuss some strategies for using them effectively. In particular, we are going to focus on *semantic versioning* and a "branching strategy" called *Git-flow*. Both of these ideas are widely used and extremely powerful for managing software projects.

Semantic Versioning

We live in a world of version numbers – it's hard to miss it. From physical device launches to software updates to game releases, these numbers are almost universally understood to mean "something has changed" when the numbers increase. In the world of software development, there is a special scheme used for assigning version numbers called **semantic versioning**.

Semantic versioning (SemVer) is a standardized system for assigning version numbers to software, encoding the version by a three-part number and other optional tags. The full system is quite in-depth (see https://semver.org/), and we aren't yet concerned with all the detail and nuance it offers. For now, we will simply incorporate some very useful ideas from the system into our toolbox to use with the branching strategy in the next section.

The SemVer Format

SemVer has a simple format with three numeric identifiers.

```
Major.Minor.Patch
```

- **Major version**. Increments with incompatible API changes or major shifts, signaling a break from backward compatibility.
- **Minor version**. Increments with new, backward-compatible features, indicating enhancement without breaking previous functionalities.
- **Patch version**. Increments with backward-compatible bug fixes that address incorrect behavior without altering functionality.

Additionally, the system includes ways to indicate pre-releases and build metadata, but we will stick to the three simple numbers for now.

SemVer Applications

SemVer is commonly used for many types of software – a few examples[1] follow to show what it can look like in different settings.

Games

- Major releases (e.g., "The Witcher 3" to "The Witcher 4"): signify complete overhauls, new game mechanics, or sequels

- Minor releases (e.g., new map in “Among Us”): introduce new features such as maps, modes, or substantial content that don’t refine the entire game
- Patches (e.g., bug fixes in “Cyberpunk 2077”): address performance improvements, bug fixes, or minor tweaks

Apps

- Major releases (e.g., Instagram’s introduction of Reels): reflect overhauls or new functionalities that change the core experience
- Minor releases (e.g., new filter options in a photo app): add new features or enhancements that are backward compatible
- Patches (e.g., security patch in WhatsApp): focus on bug fixes, security patches, or minor enhancements

Web Development

- Major releases (e.g., complete redesign of Airbnb’s website): correspond to overhauls in design, user experience, or major functionality changes
- Minor releases (e.g., new blog section on a new site): include new sections, features, or significant content updates
- Patches (e.g., security updates to a CMS): comprise fixes and security updates

GIT-FLOW

WHAT IS A BRANCHING STRATEGY?

“a strategy that software development teams adopt for writing, merging and deploying code with the help of a version control system like Git.” (https://www.geeksforgeeks.org/branching-strategies-in-git/)

The most widely used include:

- Git-flow
- GitHub-flow
- Trunk-based

Other variants and hybrid models exist. We will focus on Git-flow this quarter.

WHY GIT-FLOW?

- Git-flow requires a good working knowledge of Git to be used effectively and will serve us well as we practice using version control.
- Responsibilities are extremely clear for each branch
- Version histories are clear and easy to navigate
- Doesn’t rely on automated testing to maintain a working code base

- Works well for scheduled, predictable release schedules
- Requires good communication between team members to function well

Other strategies are overtaking git-flow in popularity in some settings that require continuous integration of new features into live software. These strategies heavily rely on automated testing infrastructure to be built and used by the development team, and (for now) those are out of scope for this course.

Strategy Overview

Git-flow requires us to have a central repository (we'll call it **`origin`**) with two permanent main branches:

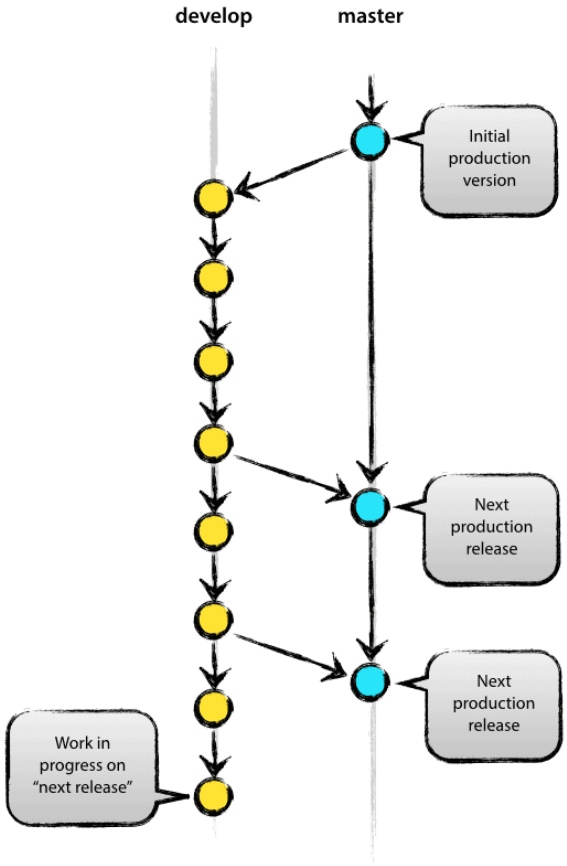
- **`master`**
- **`develop`**

The **`master`** branch at **`origin`** is the main "production-ready" code, whose latest commit reflects the most recent stable version of the code. The **`develop`** branch at **`origin`** is the "integration branch" where development changes are sent to be included in the next release.

When the source code in the dev branch reaches a stable point and is ready to be released, all of the changes should be merged back into master and tagged with a release number. We will discuss how to do this later.

Next to the two main branches are a variety of supporting branches with limited lifetimes that will eventually be removed. These are:

- **`feature`** branches
- **`release`** branches
- **`hotfix`** branches



	`feature` branches	**`release`** branches	**`hotfix`** branches
May branch off from	**`develop`**	**`develop`**	**`master`**
Must merge back into	**`develop`**	**`develop`** and **`master`**	**`develop`** and **`master`**
Branch naming convention	anything that doesn't match other branch naming conventions	**`release-*`**	**`hotfix-*`**

Feature Branches

Feature branches are used to develop new features for the upcoming (or distant future) release. When starting development of a feature, the target release in which this feature will be incorporated may well be unknown at that point. The essence of a feature branch is that it exists as long as the feature is in development, but will eventually be merged back into **`develop`** (to definitely add the new feature to the upcoming release) or discarded (in case of a disappointing experiment).

CREATING A FEATURE BRANCH

When starting work on a new feature, branch off from the **develop** branch.

```
$ git checkout -b myfeature develop
Switched to a new branch "myfeature"
```

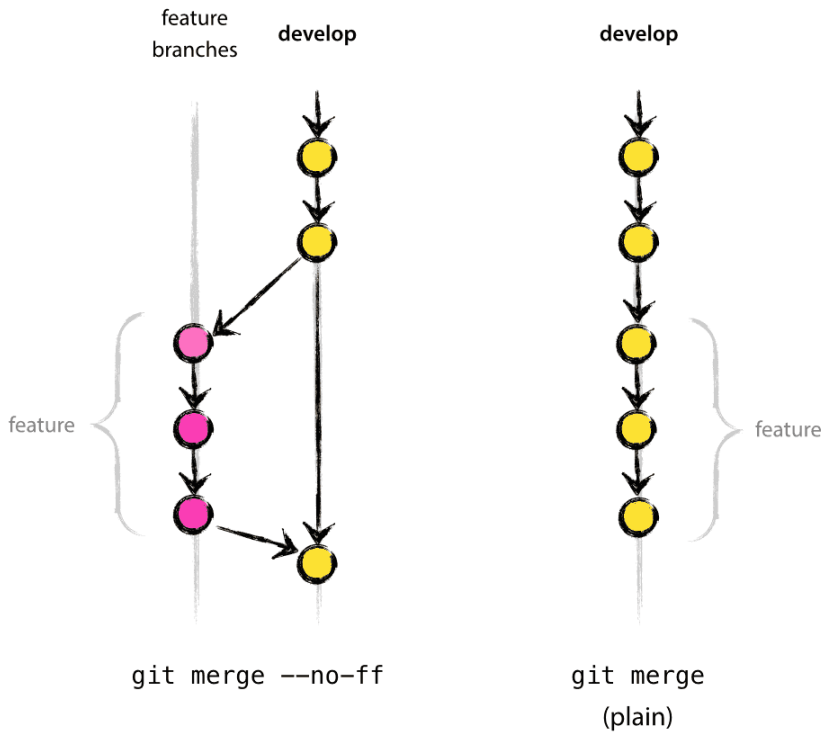
INCORPORATING A FINISHED FEATURE ON DEVELOP

Finished features may be merged into the **develop** branch to definitely add them to the upcoming release:

```
$ git checkout develop
Switched to branch 'develop'
$ git merge --no-ff myfeature
Updating ea1b82a..05e9557
(Summary of changes)
$ git branch -d myfeature
Deleted branch myfeature (was 05e9557).
$ git push origin develop
```

The **--no-ff** flag causes the merge to always create a new commit object, even if the merge could be performed with a fast-forward. This avoids losing information about the historical existence of a feature branch and groups together all commits that together added the feature. Compare:

In the latter case, it is impossible to see from the Git history which of the commit objects together have implemented a feature—you would have to manually read all the log messages. Reverting a



whole feature (i.e. a group of commits), is a true headache in the latter situation, whereas it is easily done if the **`--no-ff`** flag was used.

Yes, it will create a few more (empty) commit objects, but the gain is much bigger than the cost.

Release Branches

Release branches support preparation of a new release. They allow for last-minute clean-up, minor bug fixes, and preparing meta-data for the release (version number, build dates, etc.). By doing all of this work on a release branch, the **`develop`** branch is cleared to receive features for the next big release.

The key moment to branch off a new release branch from **`develop`** is when develop (almost) reflects the desired state of the new release. At least all features that are targeted for the upcoming release must be merged into **`develop`** at this point in time. All other features may not – they must wait until after the release branch is branched off.

It's exactly at the start of a release branch that the upcoming release gets assigned a version number – not any earlier (more on this later). Up until that moment, the **`develop`** branch reflected changes for the "next release", but it is unclear whether that next release will eventually become 0.3 or 1.0 (for example) until the release branch is started. That decision is made on the start of the release branch and is carried out by the project's rules on version number bumping.

Creating a release branch

Release branches are created from the **`develop`** branch. For example, say version 1.1.5 is the current production release and we have a big release coming up. The state of **`develop`** is ready for the "next release" and we have decided that this will become version 1.2 (rather than 1.1.6 or 2.0). So we branch off and give the release branch a name reflecting the new version number:

```
$ git checkout -b release-1.2 develop
Switched to a new branch "release-1.2"
$ ./bump-version.sh 1.2
Files modified successfully, version bumped to 1.2.
$ git commit -a -m "Bumped version number to 1.2"
[release-1.2 74d9424] Bumped version number to 1.2
1 files changed, 1 insertions(+), 1 deletions(-)
```

After creating a new branch and switching to it, we bump the version number. Here, **`bump-version.sh`** is a fictional shell script that changes some files in the working copy to reflect the new version. (This can of course be a manual change—the point being that some files change.) Then, the bumped version number is committed.

This new branch may exist there for a while, until the release may be rolled out definitely. During that time, bug fixes may be applied in this branch (rather than on the develop branch). Adding large new features here is strictly prohibited. They must be merged into **`develop`**, and therefore, wait for the next big release.

FINISHING A RELEASE BRANCH

When the state of the release branch is ready to become a real release, some actions need to be carried out. First, the release branch is merged into **`master`** (since every commit on **`master`** is a new release by definition, remember). Next, that commit on **`master`** must be tagged for easy future reference to this historical version. Finally, the changes made on the release branch need to be merged back into **`develop`**, so that future releases also contain these bug fixes.

The first two steps in Git:

```
$ git checkout master
Switched to branch 'master'
$ git merge --no-ff release-1.2
Merge made by recursive.
(Summary of changes)
$ git tag -a 1.2
```

The release is now done, and tagged for future reference. To keep the changes made in the release branch, we need to merge those back into **`develop`**, though. In Git:

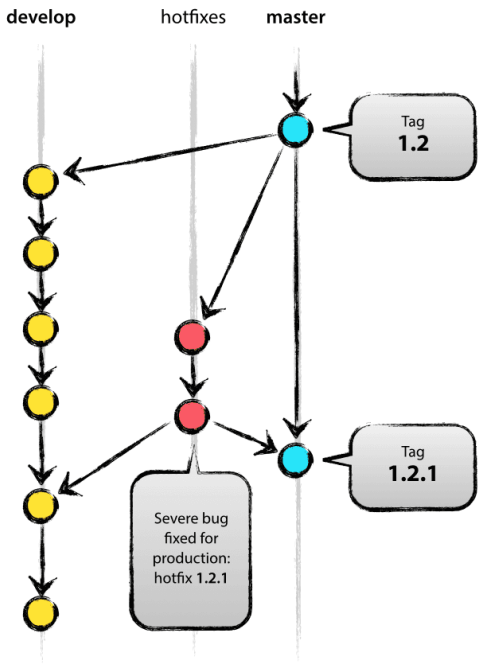
```
$ git checkout develop
Switched to branch 'develop'
$ git merge --no-ff release-1.2
Merge made by recursive.
(Summary of changes)
```

This step may well lead to a merge conflict (probably even, since we have changed the version number). If so, fix it and commit.

Now we are really done and the release branch may be removed, since we don't need it anymore:

```
$ git branch -d release-1.2
Deleted branch release-1.2 (was ff452fe).
```

HOTFIX BRANCHES



Hotfix branches are also meant to prepare for a new release, albeit unplanned. They arise from the necessity to act immediately upon an undesired state of a live production version. When a critical bug in a production version must be resolved immediately, a hotfix branch may be branched off from the corresponding tag on the master branch that marks the production version.

The essence is that work of team members (on the **`develop`** branch) can continue, while another person is preparing a quick production fix.

CREATING A HOTFIX BRANCH

Hotfix branches are created from the **`master`** branch. For example, say version 1.2 is the current production release running live and causing troubles due to a severe bug. But changes on **`develop`** are yet unstable. We may then branch off a hotfix branch and start fixing the problem:

```
$ git checkout -b hotfix-1.2.1 master
Switched to a new branch "hotfix-1.2.1"
$ ./bump-version.sh 1.2.1
Files modified successfully, version bumped to 1.2.1.
$ git commit -a -m "Bumped version number to 1.2.1"
[hotfix-1.2.1 41e61bb] Bumped version number to 1.2.1
1 files changed, 1 insertions(+), 1 deletions(-)
```

Don't forget to bump the version number after branching off! Then, fix the bug and commit the fix in one or more separate commits.

```
$ git commit -m "Fixed severe production problem"
[hotfix-1.2.1 abbe5d6] Fixed severe production problem
5 files changed, 32 insertions(+), 17 deletions(-)
```

FINISHING A HOTFIX BRANCH

When finished, the bugfix needs to be merged back into **`master`**, but also needs to be merged back into **`develop`**, in order to safeguard that the bugfix is included in the next release as well. This is completely similar to how release branches are finished.

First, update **`master`** and tag the release.

```
$ git checkout master
Switched to branch 'master'
$ git merge --no-ff hotfix-1.2.1
Merge made by recursive.
(Summary of changes)
$ git tag -a 1.2.1
```

Next, include the bugfix in **develop**, too:

```
$ git checkout develop
Switched to branch 'develop'
$ git merge --no-ff hotfix-1.2.1
Merge made by recursive.
(Summary of changes)
```

The one exception to the rule here is that, when a release branch currently exists, the hotfix changes need to be merged into that release branch, instead of **develop**. Back-merging the bugfix into the release branch will eventually result in the bugfix being merged into **develop** too, when the release branch is finished. (If work in **develop** immediately requires this bugfix and cannot wait for the release branch to be finished, you may safely merge the bugfix into **develop** now already as well.)

Finally, remove the temporary branch:

```
$ git branch -d hotfix-1.2.1
Deleted branch hotfix-1.2.1 (was abbe5d6).
```